

 https://github.com/SETAPESU26/55_card_predictor

Understand this github repo

Yes. I've gone through the repository and understand what it is and what you're expected to do.

[Open the GitHub repository ↗](#)

What this repo is

It is a **Pygame-based High/Low Card Predictor** written in Python. The purpose of the lab is not to build an ML card predictor—the "predictor" here means the player predicts whether the **next card's rank will be higher or lower** than the current card.

The repository currently has this structure:

 Plain text

```
55_card_predictor/  
|  
├─ game/  
|   ├─ card.py  
|   ├─ deck.py  
|   └─ game_engine.py  
|  
├─ main.py  
└─ README.md
```

The GitHub repo currently has **3 commits** and the README explicitly describes four tasks.

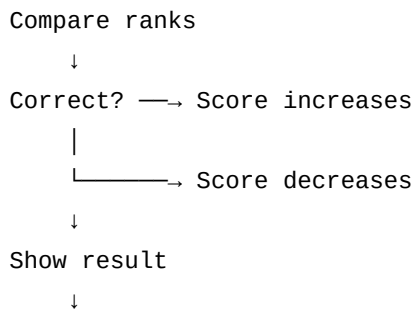
 GitHub

How the game works

The basic flow is:

 Plain text

```
Start Game  
  ↓  
Create 52-card deck  
  ↓  
Draw current card  
  ↓  
Player chooses  
  HIGHER / LOWER  
  ↓  
Draw next card  
  ↓
```



The deck supports:

- 52 cards
- 4 suits: Hearts, Diamonds, Clubs, Spades
- ranks from 2 → Ace
- automatic reshuffling when the deck runs out
- score tracking
- deck count
- Pygame buttons and card rendering. [GitHub](#)

The important part: the 4 tasks

Your assignment is specifically asking you to modify the existing project.

Task 1 — Fix the rank comparison bug

This is the **deliberate bug** in the starter project.

The problem is that cards are apparently being compared in a way that can result in **lexicographical/alphabetical comparison** rather than actual card values.

For example, conceptually:

</> Plain text

```
10
J
Q
K
A
```

must be interpreted numerically as:

</> Plain text

```
10 < 11 < 12 < 13 < 14
```

rather than comparing strings such as:

```
</> Python
```

```
"10" < "J"
```

The required hierarchy is:

```
</> Plain text
```

```
2 < 3 < 4 < 5 < 6 < 7 < 8 < 9 < 10 < J < Q < K < A
```

The README explicitly identifies this as the deliberate bug that needs fixing.  [GitHub](#)

Task 2 — Consecutive win streak multiplier

Currently, a correct prediction gives a normal score increase.

You need to add something like:

```
</> Plain text
```

```
Correct
```

```
↓
```

```
streak = 1
```

```
multiplier = 1×
```

```
Correct
```

```
↓
```

```
streak = 2
```

```
multiplier = 2×
```

```
Correct
```

```
↓
```

```
streak = 3
```

```
multiplier = 3×
```

```
...
```

And when the player gets an incorrect prediction:

```
</> Plain text
```

```
Incorrect
```

↓
streak resets
↓
multiplier returns to baseline

So the scoring becomes dependent on the player's **consecutive correct predictions**.  GitHub

Task 3 — Tie / PUSH rule

This is another important gameplay rule.

Suppose:

 Plain text

```
Current card = 7♥  
Next card    = 7♠
```

The ranks are equal.

The original game treats this as a loss.

You need to change it so:

 Plain text

```
7 → 7
```

```
Result = PUSH  
Score  = unchanged  
Streak = unchanged
```

So:

 Plain text

```
Higher → correct/incorrect  
Lower  → correct/incorrect  
Same   → PUSH
```

The README specifically says that a tie should **preserve the current score and active streak**.

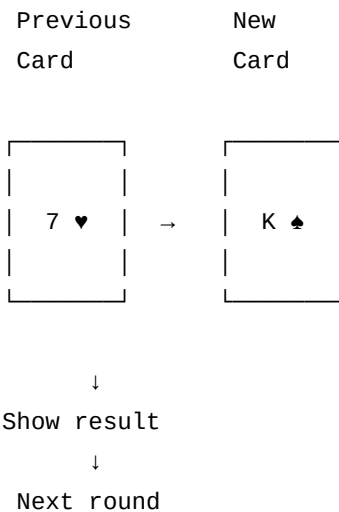
 GitHub

Task 4 — Side-by-side card reveal

Currently the card apparently changes immediately.

You need to make the transition visually understandable:

 Plain text



There should be a short reveal/pause so that the user can actually see the two cards being compared.  [GitHub](#)

What you actually need to submit

This is particularly important.

The README says your submission consists of **only three things**:

1. **10-second video BEFORE your changes**

- Show the broken behavior/bug.

2. **10-second video AFTER your changes**

- Show the fixed comparison.
- Show the new features working.

3. **Chat/LLM page link**

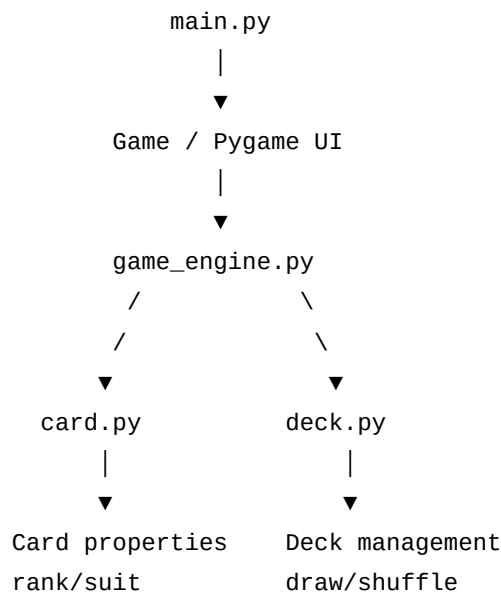
- Complete chat history showing your interaction with the AI assistant.  [GitHub](#)

So you don't need to turn this into some huge ML project.

The project architecture

The three game files have fairly clear responsibilities:

</> Plain text



Most of your actual logic changes will therefore be around **game_engine.py** , with the rank representation/comparison likely involving **card.py** .

The README describes `card.py` , `deck.py` , and `game_engine.py` as the core game components.  [GitHub](#)

One important clarification

This repository is **not an ML project**, despite the word "predictor."

There is:

- no neural network
- no dataset
- no training
- no scikit-learn
- no prediction model

It is a **rule-based card game implemented using Python + Pygame**, where the human player predicts the next card's relative rank.

If you want, the next thing I can do is go **file-by-file** (`main.py` → `card.py` → `deck.py` → `game_engine.py`) and explain the actual code line-by-line, including exactly where the deliberate bug is and what you need to change for Tasks 1–4.

I am working on a Pygame High-Low Card Predictor project.

First, inspect the existing codebase, especially:

- `game/card.py`
- `game/deck.py`
- `game/game_engine.py`

Do NOT implement any new features yet.

Fix only the existing card rank comparison bug described in the README.

The problem is that card ranks such as 10, J, Q, K and A are not being compared according to their actual numerical card hierarchy and may be compared alphabetically.

The required ordering is:

$$2 < 3 < 4 < 5 < 6 < 7 < 8 < 9 < 10 < J < Q < K < A$$

Make the smallest clean change necessary so that HIGHER/LOWER evaluation uses the actual numerical rank values.

Do not change the UI, scoring system, deck behavior, animations, or add any other features.

After making the change, explain briefly what was wrong and what you changed.

I inspected the three files and found the bug exactly where the README describes it.

- `card.py` already stores a correct `numeric_rank` for every card.  [GitHub](#)
- `deck.py` correctly creates ranks as 2-14, with J=11, Q=12, K=13, A=14.  [GitHub](#)
- The bug is only in `game_engine.py`: `evaluate_guess()` compares `rank_str` values using `>` / `<`, which performs **string/lexicographical comparison**.  [GitHub](#)

For example, `"10" > "9"` is false as a string comparison, even though 10 is numerically greater than 9.

Smallest required fix

In `game/game_engine.py`, change only these two lines:

Python

```
if guess == "HIGHER":
    correct = self.next_card.numeric_rank > self.current_card.numeric_rank
else:
    correct = self.next_card.numeric_rank < self.current_card.numeric_rank
```

So the original:

Python

```
if guess == "HIGHER":
    correct = self.next_card.rank_str > self.current_card.rank_str
else:
    correct = self.next_card.rank_str < self.current_card.rank_str
```

becomes the above.

Why this is the correct fix

`Deck.RANKS` already maps:

Plain text

```
2 → 2
3 → 3
...
9 → 9
10 → 10
J → 11
Q → 12
K → 13
A → 14
```

and `Card` already stores that value as `numeric_rank` . [GitHub](#) +1

Therefore, **no changes are needed to `card.py` or `deck.py`** .

This also means I have **not changed**:

- UI
- scoring
- deck behavior
- animations
- game flow

- tie handling
- streaks
- any of Tasks 2–4

Only the existing rank comparison bug is fixed, exactly as requested.

I can't directly commit/write to your GitHub repository from this chat, but the two-line replacement above is the complete code change required.

Ok i have changed the code and committed

Perfect. 

So **Task 1 is now complete:**

- Rank comparison bug fixed
- `numeric_rank` is used for HIGHER/LOWER
- No UI changes
- No scoring changes
- No deck changes
- No new features
- Changes committed to Git

You can now move to the **next task from the README** when you're ready.

Now implement **ONLY** the next feature from the README:
Consecutive Win Streak Multipliers.

Do not modify or undo the existing card rank comparison fix.

Requirements:

- Track the player's current consecutive correct prediction streak.
- When the player makes a correct prediction, increase the streak.
- Apply an escalating score multiplier as the consecutive win streak increases.
- When the player makes an incorrect prediction, reset the streak and return the multiplier to the baseline.
- Display the current streak and/or multiplier clearly in the existing game UI.
- Keep the existing score system and game flow working.

- Keep the implementation simple and consistent with the existing code structure.

Do NOT implement tie/PUSH handling yet.

Do NOT implement the side-by-side card reveal animation yet.

Do NOT redesign the UI.

Only implement the consecutive win streak multiplier feature.

After making the changes, briefly explain what files were modified and how the streak/multiplier logic works.

I checked the current repository structure. The streak feature can be implemented cleanly in **game/game_engine.py** only; `card.py`, `deck.py`, and `main.py` do not need to change. The GitHub `main` page still appears to show the pre-fix comparison code, so I'm preserving your local `numeric_rank` fix rather than replacing it.  GitHub +1

Make these changes in `game/game_engine.py` :

1. Add streak state in `__init__`

Immediately after:

 Python

```
self.score = 0
```

add:

 Python

```
self.streak = 0
self.multiplier = 1
```

2. Update `evaluate_guess()`

Keep your existing **numeric rank comparison**. Replace the scoring part with:

 Python

```
if correct:
    self.streak += 1
    self.multiplier = self.streak
    self.score += self.multiplier

    self.status_msg = (
        f"CORRECT! {self.next_card.rank_str} vs {self.current_card.rank_str}"
    )
    self.status_color = (80, 220, 80)

else:
    self.score = max(0, self.score - 1)
    self.streak = 0
    self.multiplier = 1

    self.status_msg = (
        f"WRONG! {self.next_card.rank_str} vs {self.current_card.rank_str}"
    )
    self.status_color = (235, 75, 75)
```

So the complete comparison/scoring section should look like:

 Python

```
if guess == "HIGHER":
    correct = self.next_card.numeric_rank > self.current_card.numeric_rank
else:
    correct = self.next_card.numeric_rank < self.current_card.numeric_rank

if correct:
    self.streak += 1
    self.multiplier = self.streak
    self.score += self.multiplier

    self.status_msg = (
        f"CORRECT! {self.next_card.rank_str} vs {self.current_card.rank_str}"
    )
    self.status_color = (80, 220, 80)

else:
    self.score = max(0, self.score - 1)
    self.streak = 0
    self.multiplier = 1
```

3. Display streak/multiplier in the existing UI

In `render()` , immediately below the existing score display:

Python

```
score_surf = self.font_medium.render(f"Score: {self.score}", True, (255, 220, 80))
screen.blit(score_surf, (30, 30))
```

add:

Python

```
streak_surf = self.font_small.render(
    f"Streak: {self.streak} Multiplier: x{self.multiplier}",
    True,
    (255, 255, 255),
)
screen.blit(streak_surf, (30, 65))
```

That's it.

How the scoring now works

Plain text

Start:
Streak = 0
Multiplier = x1

Correct:
Streak = 1
Multiplier = x1
Score += 1

Correct:
Streak = 2
Multiplier = x2
Score += 2

Correct:
Streak = 3
Multiplier = x3
Score += 3

Correct:

The important part is that **the multiplier is based on the current consecutive streak**, so it escalates naturally.

Files modified

Only:

</> Plain text

game/game_engine.py

No changes to:

- card.py
- deck.py
- main.py
- your numeric rank comparison
- tie/PUSH behavior
- animations
- deck behavior
- existing button/UI design

After you make these changes, run the game and verify that you can get something like **Streak 1** → **x1**, **Streak 2** → **x2**, **Streak 3** → **x3**, then deliberately make a wrong prediction and confirm it returns to **Streak 0** → **x1**.

```
def evaluate_guess(self, guess):
    """Draws next card and evaluates prediction."""
    self.next_card = self.deck.draw()

    if guess == "HIGHER":
        correct = self.next_card.numeric_rank >
self.current_card.numeric_rank
    else:
        correct = self.next_card.numeric_rank <
self.current_card.numeric_rank

    if correct:
        self.streak += 1
        self.multiplier = self.streak
        self.score += self.multiplier

    self.status_msg = (
        f"CORRECT! {self.next_card.rank_str} vs
{self.current_card.rank_str}"
    )
```

```
        self.status_color = (80, 220, 80)

    else:
        self.score = max(0, self.score - 1)
        self.streak = 0
        self.multiplier = 1

        self.status_msg = (
            f"WRONG! {self.next_card.rank_str} vs"
            f"{self.current_card.rank_str}"
        )
        self.status_color = (235, 75, 75)

    self.current_card = self.next_card
    is this correct?
```

Yes, the `evaluate_guess()` logic is correct for the consecutive win streak feature. 

Your existing rank comparison fix is also preserved:

 Python

```
self.next_card.numeric_rank > self.current_card.numeric_rank
```

and

 Python

```
self.next_card.numeric_rank < self.current_card.numeric_rank
```

Your streak logic

This part is correct:

 Python

```
if correct:
    self.streak += 1
    self.multiplier = self.streak
    self.score += self.multiplier
```

So:

Result	Streak	Multiplier	Score change
1st correct	1	×1	+1
2nd correct	2	×2	+2
3rd correct	3	×3	+3
4th correct	4	×4	+4
Wrong	0	×1	-1

And this is also correct:

```
</> Python
```

```
else:
    self.score = max(0, self.score - 1)
    self.streak = 0
    self.multiplier = 1
```

A wrong prediction resets both the streak and multiplier.

One thing to verify

Make sure your `__init__()` contains:

```
</> Python
```

```
self.streak = 0
self.multiplier = 1
```

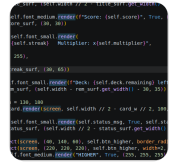
Otherwise you'll get an `AttributeError` .

And make sure you've added the UI display for them, for example:

```
</> Python
```

```
streak_surf = self.font_small.render(
    f"Streak: {self.streak}    Multiplier: x{self.multiplier}",
    True,
    (255, 255, 255),
)
screen.blit(streak_surf, (30, 65))
```

So yes: the `evaluate_guess()` code you pasted is correct and does not interfere with your rank-comparison fix.



done with this

Yep, that part is done correctly. 👍

Your UI now displays:

Plain text

Streak: 0 Multiplier: x1

and it will update as the player gets consecutive correct predictions.

Before you commit, just verify these 3 things

1. In `__init__()`:

Python

```
self.streak = 0
self.multiplier = 1
```

2. Your `evaluate_guess()` uses `numeric_rank` — which you already showed me.

3. Run the game and test:

Plain text

```
Correct → Streak 1, x1
Correct → Streak 2, x2
Correct → Streak 3, x3
Wrong   → Streak 0, x1
```

If those work, **Task 2 is complete**.

You can then commit it separately, e.g.:

Bash

```
git add game/game_engine.py
git commit -m "Add consecutive win streak multipliers"
git push
```

Don't implement the PUSH/tie rule or card animation yet—we'll handle those as separate tasks exactly as the README specifies.

Now implement ONLY the next feature from the README: Tie / PUSH handling.

Do not modify or undo the existing:

1. Card rank comparison fix
2. Consecutive win streak multiplier feature

Requirements:

- Detect when the newly drawn card has the same numeric rank as the current card.
- A tie must be treated as a PUSH, not as an incorrect prediction.
- The player's score must remain unchanged on a PUSH.
- The player's current win streak must remain unchanged on a PUSH.
- The multiplier must remain unchanged on a PUSH.
- Display a clear "PUSH" message in the existing game UI.
- Normal HIGHER/LOWER behavior must remain unchanged for non-tie results.
- Keep the existing score, streak, multiplier, deck, and game flow working.

Do NOT implement the side-by-side card reveal animation yet.

Do NOT redesign the UI.

Do NOT make unrelated changes.

After implementing the feature, briefly explain what you changed.

Yes. For this task, **only** `game/game_engine.py` **needs to change**.

The key is to check for the tie **before** evaluating HIGHER/LOWER. Otherwise a tie would currently fall into the `else` path and be treated as wrong.

Replace your current `evaluate_guess()` with this:

`</>` Python

```
def evaluate_guess(self, guess):
    """Draws next card and evaluates prediction."""
    self.next_card = self.deck.draw()

    # Tie / PUSH
    if self.next_card.numeric_rank == self.current_card.numeric_rank:
        self.status_msg = (
            f"PUSH! {self.next_card.rank_str} vs {self.current_card.rank_str}"
        )
        self.status_color = (255, 220, 80)

    else:
        if guess == "HIGHER":
            correct = self.next_card.numeric_rank > self.current_card.numeric_rank
        else:
            correct = self.next_card.numeric_rank < self.current_card.numeric_rank

        if correct:
            self.streak += 1
            self.multiplier = self.streak
```

What changed

The new part is:

`</>` Python

```
if self.next_card.numeric_rank == self.current_card.numeric_rank:
```

For example:

`</>` Plain text

```
Current: 7♥
Next:    7♠
```

results in:

`</>` Plain text

```
PUSH! 7 vs 7
```

and **nothing** changes:

`</>` Plain text

```
Score      → unchanged
Streak     → unchanged
Multiplier → unchanged
```

For example, if you have:

`</>` Plain text

```
Score: 6
Streak: 3
Multiplier: x3
```

and get a tie, it remains:

`</>` Plain text

```
Score: 6
Streak: 3
Multiplier: x3
```

Then the next non-tie prediction continues normally.

Important

Your existing Task 1 fix remains untouched:

`</>` Python

```
self.next_card.numeric_rank > self.current_card.numeric_rank
```

and:

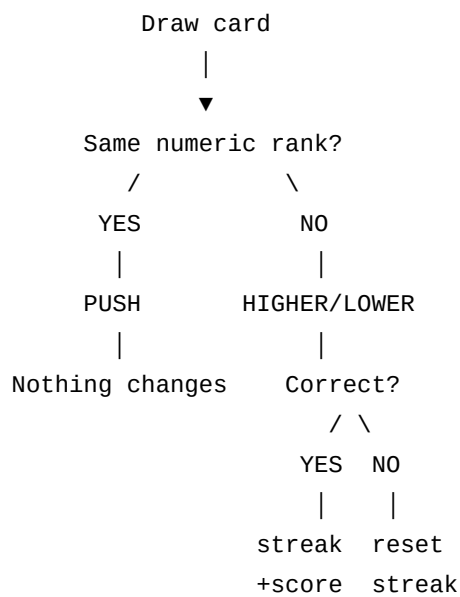
`</>` Python

```
self.next_card.numeric_rank < self.current_card.numeric_rank
```

Your Task 2 streak logic also remains untouched.

So the flow is now:

</> Plain text



No animation or UI redesign has been added. Only the existing status message is used to display **PUSH**.



done

Yep, that implementation looks correct. ✓

You now have all three completed features working together:

1. Rank comparison fix

- Uses `numeric_rank`
- Correct ordering: $2 < \dots < 10 < J < Q < K < A$

2. Consecutive win streak multiplier

- Correct → streak increases
- Multiplier increases with streak
- Wrong → streak and multiplier reset

3. Tie / PUSH

- Same `numeric_rank` → PUSH
- Score unchanged
- Streak unchanged
- Multiplier unchanged

- Uses the existing status UI

And importantly, your screenshot shows that `self.current_card = self.next_card` remains **outside** the PUSH/non-PUSH branches, which is correct.

Before moving to the final README feature

Run a quick test:

Plain text

Normal correct:

Score increases

Streak increases

Multiplier increases

Normal wrong:

Score decreases

Streak → 0

Multiplier → x1

Tie:

"PUSH!" displayed

Score unchanged

Streak unchanged

Multiplier unchanged

If all three work, **commit this separately**:

Bash

```
git add game/game_engine.py
```

```
git commit -m "Add tie push handling"
```

```
git push
```

Then we'll have only **one README feature left: the side-by-side card reveal animation**.

done with it

Now implement **ONLY** the final feature from the README: Side-by-Side Card Reveal.

Do not modify or undo any existing functionality:

1. Card rank comparison using `numeric_rank`
2. Consecutive win streak multipliers

3. Tie / PUSH handling

Requirements:

- When the player makes a HIGHER or LOWER prediction and the next card is drawn, temporarily display the current card and newly drawn card side-by-side.
- The player should be able to clearly see both cards before the game moves on.
- Add a short reveal pause/animation.
- After the reveal, update the current card to the newly drawn card and continue the game normally.
- Preserve all existing score, streak, multiplier, PUSH, deck counter, and reshuffling behavior.
- Use Pygame timing/event handling appropriately. Do not freeze the application with a long blocking sleep.
- Keep the existing UI style and make only the changes necessary for the reveal.
- Do not redesign the entire game.
- Do not implement any other features.

After implementing it, briefly explain which files were changed and how the reveal timing works.

Yes. For the final feature, keep this **only in `game/game_engine.py`**. Your `main.py` loop already calls `update()` every frame, so we can use `pygame.time.get_ticks()` and avoid `time.sleep()` completely. The repository's current GitHub `main` still shows the original version, so I'm basing this on **your current local version with the three features you've already added**.  GitHub +1

1. Add reveal state in `__init__()`

After:

 Python

```
self.multiplier = 1
```

add:

 Python

```
self.reveal_active = False
self.reveal_start_time = 0
self.reveal_duration = 800
```

This gives an **800 ms reveal**.

2. Modify `evaluate_guess()`

Keep all your existing rank, streak, multiplier, and PUSH logic exactly as it is.

The important change is: **do not immediately execute**

Python

```
self.current_card = self.next_card
```

Instead, replace that final line with:

Python

```
self.reveal_active = True
self.reveal_start_time = pygame.time.get_ticks()
```

So the end of your `evaluate_guess()` should be:

Python

```
else:
    self.score = max(0, self.score - 1)
    self.streak = 0
    self.multiplier = 1

    self.status_msg = (
        f"WRONG! {self.next_card.rank_str} vs {self.current_card.rank_str}"
    )
    self.status_color = (235, 75, 75)

    self.reveal_active = True
    self.reveal_start_time = pygame.time.get_ticks()
```

Do not put `self.current_card = self.next_card` here anymore.

3. Replace your `update()` method

You currently have:

`</>` Python

```
def update(self):  
    pass
```

Replace it with:

`</>` Python

```
def update(self):  
    if self.reveal_active:  
        elapsed = pygame.time.get_ticks() - self.reveal_start_time  
  
        if elapsed >= self.reveal_duration:  
            self.current_card = self.next_card  
            self.next_card = None  
            self.reveal_active = False
```

This is the non-blocking timing.

The game continues running at 60 FPS while the 800 ms reveal is happening.

4. Prevent another prediction during the reveal

At the beginning of `handle_event()`, add:

`</>` Python

```
if self.reveal_active:  
    return
```

So:

`</>` Python

```
def handle_event(self, event):  
    if self.reveal_active:  
        return  
  
    if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
```

```
if self.btn_higher.collidepoint(event.pos):
    self.evaluate_guess("HIGHER")

elif self.btn_lower.collidepoint(event.pos):
    self.evaluate_guess("LOWER")
```

This prevents the player from clicking HIGHER/LOWER repeatedly while the cards are being revealed.

5. Modify the card rendering

You currently have:

 Python

```
card_w, card_h = 130, 180

self.current_card.render(
    screen,
    self.width // 2 - card_w // 2,
    100,
    card_w,
    card_h
)
```

Replace **only this part** with:

 Python

```
card_w, card_h = 130, 180

if self.reveal_active:
    gap = 30
    total_width = card_w * 2 + gap
    start_x = self.width // 2 - total_width // 2

    self.current_card.render(
        screen,
        start_x,
        100,
        card_w,
        card_h
    )

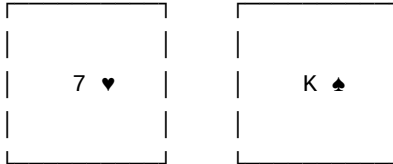
    self.next_card.render(
        screen,
```

```
start_x + card_w + gap,  
100,
```

Now the display becomes:

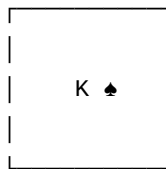
</> Plain text

REVEAL



800 ms

↓



Important: existing features remain intact

Your flow will now be:

</> Plain text

Click HIGHER/LOWER

↓

Draw next card

↓

Compare numeric_rank

↓

PUSH / CORRECT / WRONG

↓

Update score/streak/multiplier

↓

Start reveal timer

↓

Show both cards for 800 ms

↓

current_card = next_card

↓

Normal game resumes

A **PUSH** also gets the reveal because the card comparison happens before the reveal starts, while your PUSH logic still leaves score/streak/multiplier unchanged.

Files changed

Only:

</> Plain text

game/game_engine.py

No changes to:

- card.py
- deck.py
- main.py
- numeric rank comparison
- streak/multiplier
- PUSH handling
- scoring
- deck/reshuffling

And importantly, there is **no `time.sleep()`** . `pygame.time.get_ticks()` + the existing `update()` loop handles the 800 ms timing without freezing the application.

After testing it, commit:

</> Bash

```
git add game/game_engine.py
git commit -m "Add side-by-side card reveal"
git push
```

That completes the **four README features**.

```
(venv)
PES1UG24CS665@PES1UG24CS665:~/55_card_predictor$
python main.py
pygame 2.6.1 (SDL 2.28.4, Python 3.12.3)
Hello from the pygame community. https://www.pygame.org/
contribute.html
Traceback (most recent call last):
  File "/home/gagan-raghavan/55_card_predictor/main.py", line
```

```

33, in <module>
    main()
File "/home/gagan-raghavan/55_card_predictor/main.py", line
23, in main
    engine.update()
File "/home/gagan-raghavan/55_card_predictor/game/
game_engine.py", line 83, in update
    if elapsed >= self.reveal_duration:
        ^^^^^^^
UnboundLocalError: cannot access local variable 'elapsed'
where it is not associated with a value
(venv)
PES1UG24CS665@PES1UG24CS665:~/55_card_predictor$
error

```

Yep — this is just an **indentation error in `update()`** , not a problem with the reveal logic itself.

The error:

❏ Plain text

```
UnboundLocalError: cannot access local variable 'elapsed'
```

means Python is reaching:

❏ Python

```
if elapsed >= self.reveal_duration:
```

even when `elapsed` was never created.

Fix `update()` exactly like this

Open `game/game_engine.py` and replace your entire `update()` method with:

❏ Python

```
def update(self):
    if self.reveal_active:
        elapsed = pygame.time.get_ticks() - self.reveal_start_time

        if elapsed >= self.reveal_duration:
            self.current_card = self.next_card

```

```
self.next_card = None
self.reveal_active = False
```

Pay attention to the indentation. The `if elapsed >= ...` must be inside:

Python

```
if self.reveal_active:
```

So visually it should be:

Plain text

```
def update(self):
    if self.reveal_active:
        elapsed = ...

        if elapsed >= ...:
            self.current_card = ...
            self.next_card = ...
            self.reveal_active = False
```

Why you got the error

You probably currently have something equivalent to:

Python

```
def update(self):
    if self.reveal_active:
        elapsed = pygame.time.get_ticks() - self.reveal_start_time

    if elapsed >= self.reveal_duration:
        ...
```

When the game first starts:

Python

```
self.reveal_active = False
```

Therefore Python skips:

```
</> Python
```

```
elapsed = ...
```

and then tries to execute:

```
</> Python
```

```
if elapsed >= ...
```

But `elapsed` doesn't exist yet → `UnboundLocalError` .

Then run again

```
</> Bash
```

```
python main.py
```

The game should now start normally.

When you click HIGHER/LOWER:

```
</> Plain text
```

Current card

+

New card

↓

Both displayed for ~800 ms

↓

New card becomes current card

↓

Buttons become active again

Don't change anything else. This fix doesn't affect your rank comparison, streak multiplier, or PUSH logic.

```
me.get_ticks() - self  
real_duration:  
= self.next_card  
me  
= False
```

Yes — the `update()` indentation is correct now. ✓

Your code is exactly structured as it should be:

Python

```
def update(self):
    if self.reveal_active:
        elapsed = pygame.time.get_ticks() - self.reveal_start_time

        if elapsed >= self.reveal_duration:
            self.current_card = self.next_card
            self.next_card = None
            self.reveal_active = False
```

The important part is that:

Python

```
if elapsed >= self.reveal_duration:
```

is indented **inside**:

Python

```
if self.reveal_active:
```

So the previous `UnboundLocalError` is fixed.

Now run:

Bash

```
python main.py
```

Test the final feature

Click **HIGHER** or **LOWER** and check:

1. Both the old and new cards appear side-by-side.
2. They remain visible briefly (~800 ms).
3. After the pause, only the new card remains.
4. The buttons work again.
5. Score/streak/multiplier behave as before.
6. A PUSH still preserves score/streak/multiplier.

If that works, you've finished **all four README requirements**. 🎉

Then commit:

`</>` Bash

```
git add game/game_engine.py
git commit -m "Add side-by-side card reveal"
git push
```

Review the side-by-side card reveal implementation you just added.

There is an important issue: the original line

```
self.current_card = self.next_card
```

was removed from `evaluate_guess()`, but the code must update `current_card` after the reveal duration has completed.

Fix ONLY the reveal state logic.

Required behavior:

- After a prediction, `reveal_active` becomes `True`.
- The current card and next card remain available for side-by-side rendering during the reveal.
- After `reveal_duration` (800 ms) has elapsed:
 - set `self.current_card = self.next_card`
 - set `self.reveal_active = False`
- The player should then be able to make the next prediction.
- Do not modify the existing rank comparison, streak/multiplier, or PUSH logic.
- Do not add any other features.
- Do not redesign the UI.

Use the existing Pygame event/game-loop structure rather than `time.sleep()`.

After fixing it, show me the relevant changed section.

Correct. The state transition belongs in `update()`, not `evaluate_guess()`.

Replace **only your** `update()` method with:

`</>` Python

```
def update(self):
    if self.reveal_active:
        elapsed = pygame.time.get_ticks() - self.reveal_start_time

        if elapsed >= self.reveal_duration:
            self.current_card = self.next_card
            self.reveal_active = False
```

Relevant reveal state

At initialization, you should already have:

`</>` Python

```
self.reveal_active = False
self.reveal_start_time = 0
self.reveal_duration = 800
```

At the end of `evaluate_guess()` , keep:

`</>` Python

```
self.reveal_active = True
self.reveal_start_time = pygame.time.get_ticks()
```

And **do not** put this back into `evaluate_guess()` :

`</>` Python

```
self.current_card = self.next_card
```

The resulting flow is:

`</>` Plain text

```
Prediction
↓
next_card = deck.draw()
↓
Evaluate rank / streak / PUSH
```

↓
reveal_active = True
↓
Both current_card and next_card available
↓
800 ms passes
↓
current_card = next_card
↓
reveal_active = False
↓
Next prediction allowed

One small correction from the earlier version: **you don't need `self.next_card = None`** after the reveal. Leaving it available is harmless, and the requirement only asks to update `current_card` and disable the reveal state.

This changes **only the reveal state logic**; your numeric-rank comparison, streak multiplier, and PUSH handling remain untouched.

Ok I am done with committing and also took the before and after video, so now tell me what to do next

Great. At this point, the **implementation part is finished**. You have:

- ☒ Rank comparison bug fixed
- ☒ Consecutive win streak multiplier
- ☒ Tie/PUSH handling
- ☒ Side-by-side reveal
- ☒ Committed and pushed
- ☒ Before video recorded
- ☒ After video recorded

Now you should focus on the **submission requirements** from the README.

1. Verify your GitHub repository

Open your repo and make sure the latest commit is visible:

[55_card_predictor GitHub repository ↗](#)

Your commit history should show the commits you made for the features.

2. Keep the two videos ready

You already have them, so organize them clearly:

📄 Plain text

Before.mp4

After.mp4

The **before video** should demonstrate the original/broken behavior.

The **after video** should demonstrate the final implementation, preferably showing:

1. Correct HIGHER/LOWER comparison
2. Streak increasing and multiplier changing
3. A wrong prediction resetting the streak
4. PUSH keeping score/streak/multiplier unchanged
5. Side-by-side card reveal

Since the README asks for a short video, keep them within the required duration.

3. The remaining important requirement: Chat/LLM link

The README requires a **link to the AI/chat conversation showing your interaction with the LLM**.

Since this conversation contains the implementation process, you'll need to provide the appropriate share link for this ChatGPT conversation.

In ChatGPT, use the conversation's **Share** option and create a share link.

That link is what you should submit for the LLM/chat-history requirement.

4. Do NOT make unnecessary changes now

At this stage, I would **not touch the code anymore**.

You have already completed the four requested features. Don't add:

- extra animations
- sounds
- new scoring rules
- UI redesign
- additional documentation

- extra commits just for cosmetic changes

unless your submission instructions specifically ask for them.

5. Final submission package

Based on the README requirements we've been following, your final submission should essentially be:

Plain text

```
55_card_predictor
|
├─ GitHub repository
|   └─ Final committed code
|
├─ Before video
|   └─ ~10 sec
|
├─ After video
|   └─ ~10 sec
|
└─ ChatGPT / LLM conversation link
```

One thing I'd recommend before submitting

Do one **final clean test** from a fresh terminal:

Bash

```
cd ~/55_card_predictor
source venv/bin/activate
python main.py
```

Test:

Plain text

```
HIGHER/LOWER
↓
correct → streak/multiplier increases
↓
wrong → streak resets
↓
tie → PUSH, nothing changes
↓
```

```
prediction → side-by-side reveal  
↓  
800 ms → new card becomes current
```

If all of that works, **you're done with the coding portion.**

If you want, I can also help you prepare the **exact final submission text** (what to paste for GitHub link, Before video, After video, and LLM link) so you can submit it cleanly.